

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH



BÁO CÁO BÀI TẬP LỚN

MÔN: THIẾT KẾ CƠ SỞ DỮ LIỆU

Đề tài: Thiết kế Cơ sở Dữ liệu cho Nền tảng Dịch vụ Tín dụng & Điểm thưởng cấp doanh nghiệp (OctaPoint CaaS)

Nhóm sinh viên thực hiện:

STT	Họ và tên	Lớp
1	Lưu Minh Hoà	Lớp: 012012100203
2	Lương Tiến Cường	Lớp: 012012100203
3	Trần Thị Ngọc Duyên	Lớp: 012012100203
4	Vũ Văn Hiếu	Lớp: 012012100203
5	Trần Văn Dương	Lớp: 012012100203

TP. Hồ Chí Minh, Năm 2026

Mục lục

1	Khảo sát Yêu cầu và Nghiệp vụ Hệ thống	2
1.1	Tổng quan về bài toán và phạm vi đề tài	2
1.1.1	Bối cảnh và mục tiêu	2
1.1.2	Phạm vi báo cáo bốn chương	2
1.1.3	Đối tượng sử dụng và Yêu cầu chức năng	2
1.1.4	Yêu cầu phi chức năng và Tác động đến thiết kế CSDL	4
1.1.5	Giới hạn phạm vi đề án	5
1.1.6	Giới hạn phạm vi đề án	5
1.1.7	Quy tắc nghiệp vụ trọng yếu và Ảnh xạ dữ liệu	6
2	Thiết kế Mô hình Dữ liệu Quan niệm	8
2.1	Xác định Các Thực thể và Thuộc tính	8
2.1.1	Phân hệ Tổ chức và Phân quyền (Tenant & RBAC)	8
2.1.2	Phân hệ Tín dụng và Điểm thưởng (Loyalty Engine)	9
2.1.3	Phân hệ Sổ cái Giao dịch và Kiểm toán (Ledger & Audit)	10
2.2	Xác định các mối quan hệ (Relationships)	10
2.2.1	Sơ đồ thực thể liên kết (ERD)	12
3	Thiết kế Cơ sở dữ liệu Logic và Chuẩn hóa	13
3.1	Chuyển đổi từ Mô hình Quan niệm sang Lược đồ Logic	13
3.1.1	Các quy tắc chuyển đổi áp dụng	13
3.1.2	Danh sách Lược đồ Quan hệ sơ bộ	14
3.2	Phụ thuộc hàm và Khóa ứng viên	14
3.2.1	Bao đóng và tính tối thiểu	15
3.3	Chuẩn hóa Dữ liệu	15
3.3.1	Cách kiểm tra dạng chuẩn	16
3.4	Sơ đồ Lược đồ Logic (Logical Schema Diagram)	16
3.4.1	Sơ đồ thiết kế Logic	16
3.5	Từ điển Dữ liệu (Data Dictionary)	17
3.5.1	Bảng TENANT (Đơn vị thuê bao)	17
3.5.2	Bảng MERCHANT (Doanh nghiệp đối tác)	17
3.5.3	Bảng ROLE (Vai trò)	17
3.5.4	Bảng USER (Tài khoản nhân sự)	17
3.5.5	Bảng API_CREDENTIAL (Khóa tích hợp)	18
3.5.6	Bảng CAMPAIGN (Chiến dịch)	18
3.5.7	Bảng MERCHANT_CAMPAIGN (Phạm vi áp dụng)	18
3.5.8	Bảng REWARD_RULE (Quy tắc tính điểm)	18
3.5.9	Bảng CUSTOMER (Khách hàng)	18

3.5.10	Bảng CREDIT (Số dư ví điểm)	20
3.5.11	Bảng CREDIT_TRANSACTION (Số cái giao dịch điểm)	20
4	Thiết kế Vật lý và Triển khai	21
4.0.1	Chọn hệ quản trị cơ sở dữ liệu	21
4.1	Ánh xạ Kiểu dữ liệu và Ràng buộc	22
4.2	Chiến lược Đánh chỉ mục	23
4.2.1	Chỉ mục triển khai	23
4.2.2	Đánh giá và giới hạn	23
4.2.3	Mã lệnh khởi tạo cơ sở dữ liệu (T-SQL)	23
4.3	Tối ưu hóa truy xuất dữ liệu	26
4.3.1	Bài toán truy vấn	26
4.3.2	Cây truy vấn trước và sau khi tối ưu	26
4.3.3	Câu lệnh SQL tương ứng	26
4.3.4	Đánh giá thuật toán kết nối	27
4.4	Một số câu truy vấn mẫu	27
4.4.1	Truy vấn 1: Xem lịch sử giao dịch điểm của một khách hàng tại một cửa hàng	27
4.4.2	Truy vấn 2: Tra cứu số dư điểm hiện tại của khách hàng tại các cửa hàng đang hoạt động	27
4.4.3	Truy vấn 3: Thống kê tổng số điểm đã phát hành theo từng chiến dịch có phát sinh tích điểm trong tháng hiện tại	28

Chương 1

Khảo sát Yêu cầu và Nghiệp vụ Hệ thống

1.1 Tổng quan về bài toán và phạm vi đề tài

1.1.1 Bối cảnh và mục tiêu

OctaPoint là nền tảng dịch vụ tín dụng và điểm thưởng cho doanh nghiệp. Trong bài này, credit được hiểu là điểm thưởng của chương trình khách hàng thân thiết. Khách hàng tích điểm khi thực hiện hoạt động hợp lệ và sử dụng điểm theo quy định của cửa hàng.

Đề xuất hệ thống đầy đủ có Blockchain, SDK và MCP. Báo cáo môn Thiết kế cơ sở dữ liệu tập trung vào việc tổ chức dữ liệu quan hệ, xác định khóa và ràng buộc, chuẩn hóa và viết truy vấn mẫu. Báo cáo không trình bày các thành phần tích hợp như sản phẩm đã triển khai.

1.1.2 Phạm vi báo cáo bốn chương

1. Chương 1: mô tả bài toán, đối tượng sử dụng và yêu cầu dữ liệu.
2. Chương 2: mô hình quan niệm, thực thể, thuộc tính và mối quan hệ.
3. Chương 3: chuyển đổi sang mô hình quan hệ, phân tích khóa, chuẩn hóa và từ điển dữ liệu.
4. Chương 4: chọn hệ quản trị, kiểu dữ liệu, ràng buộc, chỉ mục và truy vấn SQL.

Bài chọn SQL Server làm hệ quản trị. Các phần Blockchain, SDK, MCP, bộ đệm, thanh toán và giám sát chỉ là bối cảnh mở rộng; không được triển khai trong phạm vi bài này.

1.1.3 Đối tượng sử dụng và Yêu cầu chức năng

Hệ thống OctaPoint CaaS phục vụ bốn nhóm tác nhân chính: Quản trị viên hệ thống (System Administrator), Doanh nghiệp đối tác (Merchant Operator), Người dùng cuối (End-User) và Tác tử AI / Hệ thống tự động (AI Agent & SDK System). Việc tách biệt rõ ràng các tác nhân giúp xác định phạm vi truy cập và thiết lập mô hình phân quyền (RBAC) chặt chẽ trong CSDL.

Hình 1.1: Sơ đồ Use Case tổng quan hệ thống OctaPoint

Quản trị viên Hệ thống (System Administrator)

FA-1. **Quản lý tài khoản và phân quyền đa khách thuê (Multi-tenant):** Tạo mới, phê duyệt, cập nhật trạng thái hoạt động của các doanh nghiệp (Merchants) đăng ký sử dụng nền tảng.

FA-2. **Giám sát hạ tầng Blockchain và Tài nguyên:** Theo dõi ví tài trợ phí gas (Enoki gas sponsorship treasuries), giám sát thông lượng API Gateway và trạng thái hợp đồng thông minh trên mạng lưới Sui.

FA-3. **Quản lý gói cước (Subscription & Billing):** Cấu hình các gói dịch vụ (Tiers), tích hợp với Stripe để quản lý thanh toán tự động và chu kỳ gia hạn của doanh nghiệp.

FA-4. **Truy vấn nhật ký kiểm toán toàn cục:** Theo dõi và truy vết (audit trail) mọi thay đổi cấu hình quan trọng trên nền tảng nhằm đảm bảo tính minh bạch.

Doanh nghiệp Đối tác (Merchant Operator)

FM-1. **Quản lý API Key và Webhook:** Khởi tạo, luân chuyển (rotate) và vô hiệu hóa cặp khóa API/Secret Key phục vụ tích hợp SDK; cấu hình điểm cuối (endpoints) nhận Webhook.

FM-2. **Cấu hình Chiến dịch & Quy tắc (Rule Engine):** Thiết lập tỷ lệ tích lũy/quy đổi điểm, hệ số nhân theo sự kiện (multiplier campaigns), điều kiện hạng thành viên (Tiers) và thời hạn sử dụng điểm.

FM-3. **Giám sát giao dịch và Đối soát:** Theo dõi luồng điểm phát hành, điểm tiêu lũy, lịch sử giao dịch khách hàng và xuất báo cáo đối soát tài chính định kỳ.

Người dùng cuối (End-User)

FU-1. **Xác thực không ví (Walletless Authentication):** Đăng nhập vào hệ thống thông qua các định danh Web2 (Google, Apple OAuth) được hỗ trợ bởi giao thức zkLogin mà không cần quản lý private key.

FU-2. **Truy vấn số dư và Lịch sử:** Xem số dư điểm thưởng hiện tại tại các Merchant khác nhau và kiểm tra lịch sử giao dịch tích/đổi điểm.

FU-3. **Thực thi giao dịch điểm thưởng:** Thực hiện các giao dịch nhận điểm hoặc thanh toán/đổi quà bằng điểm tại điểm bán (POS) hoặc ứng dụng thương mại điện tử.

Tác tử AI và Hệ thống tự động (AI Agent & SDK System)

FS-1. **Truy vấn ngữ cảnh tự động (MCP Context Retrieval):** Thông qua giao thức MCP, tự động truy xuất dữ liệu số dư, lịch sử giao dịch và quy định chiến dịch để phục vụ ngữ cảnh cho các LLM/Chatbot chăm sóc khách hàng.

FS-2. **Ghi nhận giao dịch thời gian thực:** Gửi các yêu cầu cộng/trừ điểm thưởng thông qua SDK (gRPC/REST) với độ trễ thấp (Sub-100ms).

FS-3. **Đồng bộ trạng thái On-chain/Off-chain:** Lắng nghe và đồng bộ trạng thái giao dịch từ Sui blockchain về cơ sở dữ liệu quan hệ cục bộ (SQL Server).

Bảng 1.1 – Tổng hợp yêu cầu chức năng theo vai trò:

Mã	Vai trò	Yêu cầu chức năng
FA-1	Administrator	Quản lý tài khoản đa khách thuê (Chuỗi và Cửa hàng)
FA-2	Administrator	Giám sát hạ tầng Blockchain, phí Gas và tài nguyên
FA-3	Administrator	Quản lý gói cước (Subscription), thanh toán định kỳ
FA-4	Administrator	Truy vấn nhật ký kiểm toán (Audit Trail) toàn cục
FM-1	Merchant	Quản lý API Key, Secret và cấu hình Web-hook
FM-2	Tenant/Merchant	Thiết lập chiến dịch, tỷ lệ quy đổi và phạm vi áp dụng
FM-3	Merchant	Giám sát lịch sử giao dịch tại cửa hàng, đối soát
FU-1	End-User	Đăng nhập Web2 qua zkLogin (Không dùng ví Crypto)
FU-2	End-User	Tra cứu số dư ví chung của chuỗi và lịch sử giao dịch
FU-3	End-User	Tích điểm và tiêu điểm tại điểm bán
FS-1	AI Agent/SDK	Truy vấn ngữ cảnh tự động (MCP Context Retrieval)
FS-2	AI Agent/SDK	Ghi nhận giao dịch thời gian thực (Sub-100ms)
FS-3	AI Agent/SDK	Đồng bộ trạng thái On-chain và Off-chain

1.1.4 Yêu cầu phi chức năng và Tác động đến thiết kế CSDL

Các yêu cầu phi chức năng (Non-Functional Requirements - NFR) quy định ràng buộc về chất lượng vận hành của hệ thống CaaS. Đối với khía cạnh Thiết kế Cơ sở dữ liệu, các yêu cầu này định hướng trực tiếp đến việc lựa chọn kiểu dữ liệu, thiết kế chỉ mục (indexing) và chiến lược quản lý giao dịch.

NFR-DB-01 Toàn vẹn dữ liệu & Nhất quán (Integrity & Consistency): Dữ liệu giữa các bảng phải được duy trì nhất quán sau mỗi giao dịch.

- *Hàm ý CSDL:* Bảo đảm thiết lập đầy đủ Khóa chính (PK), Khóa ngoại (FK) với chính sách tham chiếu (NO ACTION) và các ràng buộc dữ liệu (CHECK, UNIQUE). Ghi lịch sử và cập nhật số dư trong cùng giao dịch SQL ở tầng dịch vụ. Đồng bộ với Sui cần cơ chế đối soát và xử lý thử lại riêng; giao dịch SQL không bao trùm giao dịch blockchain.

NFR-DB-02 Bảo mật & Khả năng truy vết (Security & Auditability): Chỉ người dùng có quyền mới được truy cập dữ liệu. Các thay đổi quan trọng phải có khả năng truy vết.

- *Hàm ý CSDL*: Lưu mã băm API Credential (HMAC-SHA256). Nội dung nghiệp vụ của giao dịch đã hoàn tất phải bất biến. Đây là yêu cầu của tăng dịch vụ và phân quyền, chưa được triển khai bằng trigger hay quyền INSERT-only trong DDL của đề án. AuditLog thuộc phạm vi mở rộng. Việc bổ sung SuiTxDigest sau đồng bộ phải đi qua luồng được cấp quyền riêng, không cho phép sửa Amount, loại giao dịch hay ví liên quan.

NFR-DB-03 Hiệu năng & Khả năng mở rộng (Performance & Scalability): Hỗ trợ truy vấn nhanh đối với số dư, giao dịch; đáp ứng sự gia tăng về merchant, customer.

- *Hàm ý CSDL*: Tối ưu hóa chỉ mục (B+ Tree Index) trên các bảng cột lõi hướng tới mục tiêu độ trễ truy vấn dưới 100ms (chưa đo kiểm). Lược đồ phải hỗ trợ kiến trúc Multi-tenant phân tách dữ liệu an toàn.

NFR-DB-04 Tính sẵn sàng & Khả năng bảo trì (Availability & Maintainability): Dữ liệu duy trì ổn định, cấu trúc rõ ràng, chuẩn hóa và dễ mở rộng.

- *Hàm ý CSDL*: Hệ thống CSDL phải đạt tối thiểu Dạng chuẩn 3 (3NF) hoặc Boyce-Codd (BCNF) nhằm giảm dị thường cập nhật phát sinh từ các phụ thuộc hàm được xét.

Bảng 1.1: Tổng hợp yêu cầu phi chức năng ảnh hưởng đến CSDL

Mã	Thuộc tính	Yêu cầu CSDL & Cấu hình vật lý
NFR-DB-03	Hiệu năng	Tối ưu B+ Tree Index cho độ trễ truy vấn điểm < 100ms
NFR-DB-02	Bảo mật	Lưu mã băm API Secret, thiết kế phân quyền RBAC đa khách thuê
NFR-DB-01	Tính toàn vẹn	Bảng CREDIT_TRANSACTION bất biến, lưu SuiTxDigest từ blockchain
NFR-DB-04	Khả năng mở rộng	Lược đồ 3NF/BCNF theo các phụ thuộc hàm đã xét; hiệu năng cần đo kiểm

1.1.5 Giới hạn phạm vi đề án

Mặc dù hệ thống thực tế yêu cầu quản lý đa dạng các luồng thông tin, tuy nhiên trong khuôn khổ của môn học Thiết kế Cơ sở dữ liệu, mô hình thiết kế sẽ tập trung sâu vào các thực thể cốt lõi phục vụ nghiệp vụ chính bao gồm: **Tổ chức/RBAC** (Tenant, Merchant, User, Role, API_Credential) và **Khách hàng thân thiết** (Customer, Credit, Campaign, Reward_Rule, Credit_Transaction).

1.1.6 Giới hạn phạm vi đề án

Mặc dù hệ thống thực tế yêu cầu quản lý đa dạng các luồng thông tin, tuy nhiên trong khuôn khổ của môn học Thiết kế Cơ sở dữ liệu, mô hình thiết kế sẽ tập trung sâu vào các thực thể cốt lõi phục vụ nghiệp vụ chính bao gồm: **Tổ chức/RBAC** (TENANT,

MERCHANT, USER, ROLE, API_CREDENTIAL) và **Khách hàng thân thiết** (CUSTOMER, CREDIT, CAMPAIGN, CAMPAIGN_MERCHANT, REWARD_RULE, CREDIT_TRANS)

Các nhóm dữ liệu mở rộng như:

- Phân quyền chi tiết (Permission)
- Nhật ký hệ thống và Cảnh báo gian lận (AuditLog, FraudEvent)
- Quản lý gói cước (PricingPlan, Subscription, Invoice)

thuộc phạm vi mở rộng của hệ thống. Các thực thể này được xác định ở mức yêu cầu tổng quan nhưng sẽ được giản lược khỏi Mô hình quan niệm (ERD) và Mô hình logic nhằm tập trung làm rõ luồng giao dịch tích/tiêu điểm chính của hệ thống CaaS.

Giới hạn về toàn vẹn liên bảng và bảo mật: FK chỉ kiểm tra tham chiếu tồn tại, không tự cô lập dữ liệu theo Merchant. Tầng dịch vụ phải kiểm tra quyền truy cập và Campaign của một giao dịch thuộc cùng Merchant với Credit; người quản lý thuộc phạm vi được phép và không tạo chu trình; cập nhật số dư và ghi sổ cái phải nguyên tử khi có truy cập đồng thời. Những cơ chế này chưa nằm trong script khởi tạo. Dữ liệu Customer dùng chung toàn hệ thống; ví Credit được tách theo Merchant.

1.1.7 Quy tắc nghiệp vụ trọng yếu và Ánh xạ dữ liệu

Các quy tắc nghiệp vụ (Business Rules)

Các quy tắc nghiệp vụ dưới đây có tác động trực tiếp đến việc xác định các thực thể, bản số mối quan hệ và các ràng buộc toàn vẹn trong ERD.

BR-1 Ràng buộc sở hữu Chiến dịch: Một Chuỗi bán lẻ (Tenant) có thể quản lý nhiều Chiến dịch (Campaign). Mỗi Chiến dịch bắt buộc thuộc về duy nhất một Tenant và có thể được áp dụng linh hoạt cho một, nhiều hoặc toàn bộ các Cửa hàng (Merchant) trong chuỗi thông qua bảng trung gian.

BR-2 Ràng buộc độc lập Số dư: Số dư (Credit Balance) của một Khách hàng (Customer) được quản lý tập trung theo từng Chuỗi (Tenant). Nghĩa là một Customer chỉ có một ví điểm duy nhất tại mỗi Tenant, dùng chung cho mọi cửa hàng (Merchant) trực thuộc.

BR-3 Quy tắc Bất biến Giao dịch (Ledger Immutability): Bất kỳ sự thay đổi số dư nào đều sinh ra một bản ghi giao dịch (Credit Transaction) mới, có ghi nhận rõ MerchantID nơi phát sinh. Hệ thống không cho phép sửa đổi hoặc xóa giao dịch đã hoàn tất.

BR-4 Ràng buộc Miền giá trị (Domain Constraints): Tỷ lệ quy đổi, hạn mức và số lượng credit bắt buộc phải là số thực dương (>0). Trạng thái chiến dịch phải thuộc tập giá trị định trước (DRAFT, ACTIVE, EXPIRED).

BR-5 Ràng buộc Mã băm Blockchain: Giao dịch đã đồng bộ On-chain phải lưu trữ mã băm đối soát (SuiTxDigest) dưới dạng duy nhất (UNIQUE) trên cơ sở dữ liệu để phục vụ kiểm toán.

Ánh xạ Yêu cầu vào Đối tượng Dữ liệu chính

Dựa trên các yêu cầu chức năng (FR-DB) và nhóm dữ liệu cần quản lý, bảng dưới đây tổng hợp ánh xạ sang các thực thể CSDL cốt lõi:

Bảng 1.2: Ánh xạ Yêu cầu và Nhóm dữ liệu vào Thực thể

Nhóm Dữ liệu	Mã Yêu cầu	Thực thể quản lý & Vai trò
Tổ chức & Dân cư	FR-DB-01, 03	Tenant, Merchant, Customer: Quản lý định danh và hồ sơ đối tác, khách hàng.
Phân quyền	FR-DB-02	User, Role, Permission: Kiểm soát truy cập RBAC.
Loyalty	FR-DB-04, 06	Credit, Campaign, RewardRule: Quy tắc tích điểm, đổi thưởng và thời hạn.
Giao dịch On/Off	FR-DB-05, 07	CreditTransaction: Ghi nhận biến động số dư và tham chiếu mã băm Blockchain.
Bảo mật & Audit	FR-DB-08, 09	APICredential, AuditLog, FraudEvent: Quản lý API Key, nhật ký hệ thống và chống gian lận.
Thương mại	FR-DB-10	PricingPlan, Subscription, Invoice: Quản lý gói cước và thanh toán của Merchant.

Phạm vi bảng ánh xạ: Permission, AuditLog, FraudEvent, PricingPlan, Subscription và Invoice chỉ thuộc yêu cầu mở rộng; không có trong ERD, lược đồ logic hay DDL của đồ án. Các mã FR-DB ở bảng này dùng để nhóm yêu cầu dữ liệu, không phải mã FS của bảng chức năng.

Chương 2

Thiết kế Mô hình Dữ liệu Quan niệm

2.1 Xác định Các Thực thể và Thuộc tính

Dựa trên các phân hệ chức năng và quy tắc nghiệp vụ đã khảo sát, mô hình dữ liệu quan niệm của hệ thống OctaPoint CaaS xác định các thực thể cốt lõi cùng danh sách thuộc tính chi tiết.

Để đảm bảo tính liên kết chặt chẽ, kiến trúc dữ liệu của hệ thống được quy hoạch dựa trên hai trục trung tâm:

- **Trục Doanh nghiệp (Merchant):** Đóng vai trò là gốc của cấu trúc đa khách thuê (Multi-tenant). Mọi dữ liệu về chiến dịch (Campaign), người dùng (User), cấu hình API (APICredential) đều được định tuyến và cô lập dựa trên khóa ngoại MerchantID.
- **Trục Sổ cái Giao dịch (Ledger):** Tập trung vào thực thể CREDIT_TRANSACTION. Đây là rốn vũ trụ ghi nhận mọi biến động điểm thưởng, liên kết chặt chẽ với người dùng cuối (Customer), chiến dịch (Campaign) và chuỗi khối Sui để phục vụ đối soát.

2.1.1 Phân hệ Tổ chức và Phân quyền (Tenant & RBAC)

- **Thực thể TENANT (Đơn vị thuê bao gốc):**
 - TenantID (PK): Mã định danh duy nhất của hệ thống lõi.
 - TenantName (NOT NULL): Tên định danh gốc.
 - SystemConfig (NULL): Cấu hình chung dạng JSON.
- **Thực thể MERCHANT (Doanh nghiệp đối tác):**
 - MerchantID (PK): Mã định danh doanh nghiệp.
 - TenantID (FK, NOT NULL): Liên kết tới bảng Tenant.
 - BusinessName (NOT NULL): Tên thương hiệu/doanh nghiệp.
 - Status (NOT NULL): Trạng thái hoạt động (Active/Suspended).
- **USER (Người dùng/Nhân viên):**
 - UserID (PK): Mã định danh nhân viên.

- *MerchantID* (FK, NOT NULL): Mã cửa hàng trực thuộc.
- *RoleID* (FK, NOT NULL): Mã phân quyền.
- *ManagerID* (FK, NULL): Mã người quản lý trực tiếp (tự tham chiếu).
- Email (UNIQUE): Email đăng nhập hệ thống.
- PasswordHash (NOT NULL): Mật khẩu đã được mã hóa.

- **Thực thể ROLE (Vai trò):**

- RoleID (PK): Mã định danh vai trò.
- RoleName (NOT NULL): Tên vai trò (Admin, Manager, Staff).

2.1.2 Phân hệ Tín dụng và Điểm thưởng (Loyalty Engine)

- **Thực thể CAMPAIGN (Chiến dịch điểm thưởng):**

- CampaignID (PK): Mã định danh chiến dịch.
- MerchantID (FK, NOT NULL): Doanh nghiệp sở hữu chiến dịch.
- CampaignName (NOT NULL): Tên chiến dịch.
- StartDate (NOT NULL): Ngày bắt đầu.
- EndDate (NULL): Thời điểm kết thúc chiến dịch (Có thể để trống nếu chiến dịch chạy vô thời hạn).
- Status (NOT NULL): Trạng thái (Draft/Active/Expired).

- **Thực thể CAMPAIGN_MERCHANT (Phạm vi áp dụng):**

- CampaignID (PK, FK): Tham chiếu chiến dịch.
- MerchantID (PK, FK): Tham chiếu cửa hàng được áp dụng.

- **Thực thể REWARD_RULE (Quy tắc tính điểm):**

- RuleID (PK): Mã định danh quy tắc.
- CampaignID (FK, NOT NULL): Thuộc chiến dịch nào.
- ActionType (NOT NULL): Mã hành động mở rộng (ví dụ Purchase, Birthday).
- PointsMultiplier (NOT NULL): Hệ số nhân điểm, lớn hơn 0.

- **CUSTOMER (Khách hàng):**

- CustomerID (PK): Mã định danh khách hàng nội bộ của hệ thống.
- WalletAddress (UNIQUE khi khác NULL, NULL): Địa chỉ blockchain (Sui Network) được hệ thống sử dụng để định danh thông qua cơ chế Walletless Authentication.
- Email: Địa chỉ email (Định danh Web2).

- **Thực thể CREDIT (Số dư ví điểm chung):**

- CreditID (PK): Mã định danh bản ghi số dư.

- *CustomerID* (FK, NOT NULL): Liên kết khách hàng.
- *TenantID* (FK, NOT NULL): Thuộc chuỗi (Tenant) nào.
- *AvailableBalance* (NOT NULL): Số dư khả dụng hiện tại.
- * *Ràng buộc hệ thống*: **UNIQUE(CustomerID, TenantID)** để đảm bảo một khách hàng chỉ có một ví tại một chuỗi.

2.1.3 Phân hệ Sổ cái Giao dịch và Kiểm toán (Ledger & Audit)

- **Thực thể CREDIT_TRANSACTION (Giao dịch điểm):**

- *TransactionID* (PK): Mã định danh giao dịch nội bộ.
- *CreditID* (FK, NOT NULL): Liên kết tới ví số dư.
- *CampaignID* (FK, NULL): Phát sinh từ chiến dịch nào.
- *Amount* (NOT NULL): Biến động có dấu: Earn dương, Redeem âm; Refund khác 0 và đảo dấu giao dịch gốc.
- *TransactionType* (NOT NULL): Loại (Earn/Redeem/Refund).
- *SuiTxDigest* (UNIQUE, NULL): Mã băm đối soát trên chuỗi khối Sui.
- *CreatedAt* (NOT NULL): Thời điểm giao dịch.

- **Thực thể API_CREDENTIAL (Khóa tích hợp):**

- *KeyID* (PK): Mã định danh khóa.
- *MerchantID* (FK, NOT NULL): Doanh nghiệp/Cửa hàng sở hữu khóa.
- *ApiKey* (NOT NULL, UNIQUE): Mã định danh khóa công khai.
- *SecretHash* (NOT NULL): Mã băm của Secret Key.

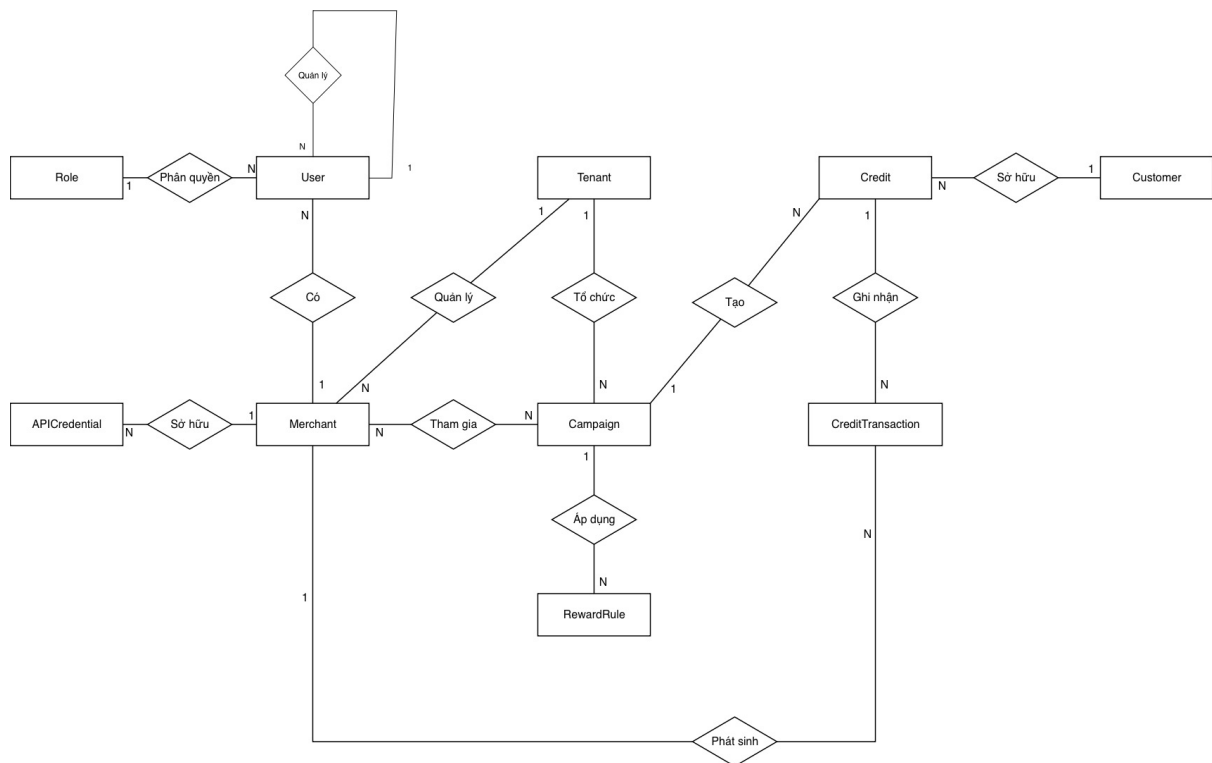
2.2 Xác định các mối quan hệ (Relationships)

Dựa trên quy trình nghiệp vụ của hệ thống OctaPoint CaaS, các mối quan hệ (Relationships) giữa các thực thể được xác định và phân tích bản số (Cardinality) chi tiết như sau:

- **Tenant - Quản lý - Merchant (1-N)**: Một tổ chức (*Tenant*) có thể quản lý nhiều đối tác kinh doanh (*Merchant*). Mỗi đối tác chỉ thuộc về một tổ chức.
- **Tenant - Tổ chức - Campaign (1-N)**: Một tổ chức (*Tenant*) có thể tổ chức nhiều chiến dịch (*Campaign*). Mỗi chiến dịch chỉ thuộc về một tổ chức.
- **Campaign - Tạo - Credit (1-N)**: Một chiến dịch (*Campaign*) phát hành/tạo ra nhiều ví/hồ sơ điểm thưởng (*Credit*). Mỗi hồ sơ điểm thưởng được cấp phát riêng dưới một chiến dịch cụ thể.
- **Customer - Sở hữu - Credit (1-N)**: Một khách hàng (*Customer*) có thể sở hữu nhiều hồ sơ điểm thưởng (*Credit*). Mỗi hồ sơ điểm thưởng thuộc về duy nhất một khách hàng.

- **Credit - Ghi nhận - CreditTransaction (1-N):** Mỗi hồ sơ điểm thưởng (*Credit*) có thể ghi nhận nhiều giao dịch (*CreditTransaction*). Mỗi giao dịch chỉ tác động lên một hồ sơ điểm.
- **Merchant - Phát sinh - CreditTransaction (1-N):** Một đối tác (*Merchant*) có thể phát sinh nhiều giao dịch điểm (*CreditTransaction*). Mỗi giao dịch chỉ do một đối tác phát sinh.
- **Campaign - Áp dụng - RewardRule (1-N):** Một chiến dịch (*Campaign*) có thể áp dụng nhiều quy tắc thưởng (*RewardRule*). Mỗi quy tắc chỉ thuộc một chiến dịch.
- **Merchant - Tham gia - Campaign (N-N):** Một đối tác (*Merchant*) có thể tham gia nhiều chiến dịch (*Campaign*) và một chiến dịch có thể có nhiều đối tác cùng tham gia.
- **Merchant - Sở hữu - APICredential (1-N):** Một đối tác (*Merchant*) sở hữu nhiều khóa xác thực (*APICredential*). Mỗi khóa chỉ cấp cho một đối tác.
- **Merchant - Có - User (1-N):** Một đối tác (*Merchant*) có nhiều người dùng/nhân viên (*User*). Mỗi nhân viên chỉ làm việc cho một đối tác.
- **Role - Phân quyền - User (1-N):** Một vai trò (*Role*) được cấp cho nhiều người dùng (*User*). Mỗi người dùng có một vai trò nhất định.
- **User - Quản lý - User (1-N):** (Quan hệ đệ quy) Một người quản lý (*Manager*) có thể quản lý nhiều nhân viên (*User*). Một nhân viên được quản lý bởi nhiều nhất một người quản lý.

2.2.1 Sơ đồ thực thể liên kết (ERD)



Hình 2.1: Sơ đồ thiết kế quan niệm (Conceptual Diagram)

CreditTransaction có khóa TransactionID riêng nên là thực thể mạnh, dù phụ thuộc tồn tại vào Credit. Quan hệ Ghi nhận không phải quan hệ định danh của thực thể yếu. Các nhãn 0..1 biểu diễn người quản lý và chiến dịch tùy chọn; mỗi cặp Customer–Merchant có tối đa một Credit.

Chương 3

Thiết kế Cơ sở dữ liệu Logic và Chuẩn hóa

Chương này chuyển mô hình quan niệm sang mô hình quan hệ, xác định khóa và kiểm tra chuẩn hóa theo các phụ thuộc hàm đã nêu. Chuẩn hóa giúp hạn chế dị thường dữ liệu; không tự bảo đảm mọi quy tắc nghiệp vụ hoặc hiệu năng truy vấn.

3.1 Chuyển đổi từ Mô hình Quan niệm sang Lược đồ Logic

Dựa trên sơ đồ thực thể kết hợp (ERD) và các quy tắc nghiệp vụ đã phân tích ở Chương 2, tiến hành chuyển đổi mô hình thực thể quan niệm sang mô hình quan hệ (Relational Schema). Quá trình này được thực hiện thông qua việc áp dụng nghiêm ngặt các thuật toán ánh xạ ER sang quan hệ.

3.1.1 Các quy tắc chuyển đổi áp dụng

Quá trình ánh xạ từ ERD sang Lược đồ Logic tuân thủ các quy tắc cốt lõi sau:

- **Quy tắc 1 (Ánh xạ Thực thể mạnh):** Mỗi thực thể (Entity) mạnh trong ERD được chuyển thành một quan hệ (Bảng - Table) trong Lược đồ Logic. Các thuộc tính đơn trở thành các cột (Columns) của bảng. Khóa định danh của thực thể trở thành Khóa chính (Primary Key - PK).
- **Quy tắc 2 (Ánh xạ mối kết hợp 1:N):** Đối với các mối kết hợp một-nhiều (ví dụ: Tenant - Merchant, Customer - Credit), đưa khóa chính của bảng phía "1" sang bảng phía "N" làm khóa ngoại (Foreign Key - FK).
- **Quy tắc 3 (Ánh xạ mối kết hợp M:N):** Tạo một quan hệ trung gian cho mỗi kết hợp nhiều-nhiều (ví dụ: Campaign - Merchant). Đưa khóa chính của hai thực thể vào làm khóa ngoại. Khóa chính của bảng trung gian thường là tổ hợp của các khóa ngoại này.

3.1.2 Danh sách Lược đồ Quan hệ sơ bộ

Dựa trên ERD của OctaPoint CaaS, áp dụng các quy tắc ánh xạ, ta thu được tập hợp các lược đồ quan hệ sơ bộ như sau. (Quy ước: Thuộc tính gạch chân là Khóa chính - PK, thuộc tính in nghiêng là Khóa ngoại - FK).

- **Role**(RoleID, RoleName)
- **Tenant**(TenantID, TenantName)
- **Customer**(CustomerID, Email, WalletAddress)
- **Merchant**(MerchantID, BusinessName, Status, *TenantID*)
- **User**(UserID, Email, PasswordHash, *RoleID*, *MerchantID*, *ManagerID*)
- **APICredential**(KeyID, ApiKey, SecretHash, *MerchantID*)
- **Campaign**(CampaignID, Status, StartDate, EndDate, *TenantID*)
- **RewardRule**(RuleID, ActionType, PointsMultiplier, *CampaignID*)
- **MerchantCampaign**(*CampaignID*, *MerchantID*)
- **Credit**(CreditID, AvailableBalance, *CustomerID*, *CampaignID*)
- **CreditTransaction**(TransactionID, Amount, CreatedAt, TransactionType, SuiTxDigest, *CreditID*, *MerchantID*)

3.2 Phụ thuộc hàm và Khóa ứng viên

Dựa trên lý thuyết thiết kế cơ sở dữ liệu quan hệ, để đảm bảo hệ thống OctaPoint không gặp phải các dị thường dữ liệu (Anomalies) khi Thêm, Xóa, Sửa, chúng ta cần phân tích tập Phụ thuộc hàm và tiến hành chuẩn hóa dữ liệu.

Xác định Tập Phụ thuộc hàm (Functional Dependencies)

Gọi U là tập hợp tất cả các thuộc tính của hệ thống. Dựa trên các quy tắc nghiệp vụ, ta xác định được tập Phụ thuộc hàm F cốt lõi như sau:

- | | |
|---|------|
| $TenantID \rightarrow TenantName, SystemConfig$ | (F1) |
| $MerchantID \rightarrow TenantID, BusinessName, Status$ | (F2) |
| $UserID \rightarrow MerchantID, Email, PasswordHash, RoleID, ManagerID$ | (F3) |
| $Email \rightarrow UserID, MerchantID, PasswordHash, RoleID, ManagerID$ | (F4) |
| $CampaignID \rightarrow TenantID, CampaignName, StartDate, EndDate, Status$ | (F5) |
| $CreditID \rightarrow CustomerID, TenantID, AvailableBalance$ | (F6) |
| $TransactionID \rightarrow CreditID, CampaignID, MerchantID, Amount, TransactionType, SuiTxDigest, CreatedAt$ | (F7) |

Giải thích nghiệp vụ:

- TENANT: $\text{TenantID} \rightarrow R$.
- MERCHANT: $\text{MerchantID} \rightarrow R$.
- ROLE: $\text{RoleID} \rightarrow R$.
- USER: $\text{UserID} \rightarrow R$; $\text{Email} \rightarrow R$, với R gồm UserID , MerchantID , Email , PasswordHash , RoleID , ManagerID .
- CUSTOMER: $\text{CustomerID} \rightarrow R$.
- CAMPAIGN: $\text{CampaignID} \rightarrow R$.
- REWARD_RULE: $\text{RuleID} \rightarrow R$.
- CREDIT: $\text{CreditID} \rightarrow R$; $(\text{CustomerID}, \text{MerchantID}) \rightarrow R$.
- API_CREDENTIAL: $\text{KeyID} \rightarrow R$; $\text{ApiKey} \rightarrow R$.
- CREDIT_TRANSACTION: $\text{TransactionID} \rightarrow R$, bao gồm cả CreatedAt .

Các khóa nghiệp vụ Email của USER, ApiKey và cặp $(\text{CustomerID}, \text{MerchantID})$ đều NOT NULL và UNIQUE. WalletAddress và SuiTxDigest chỉ duy nhất trên các dòng khác NULL; không coi chúng là khóa ứng viên của toàn quan hệ theo mô hình quan hệ cổ điển.

Không giả định RoleName , Customer.Email , CampaignName hoặc $(\text{CampaignID}, \text{ActionType})$ duy nhất. Không giả định MerchantID xác định RoleID hoặc ManagerID . Nếu bổ sung phụ thuộc hàm nghiệp vụ khác, cần kiểm tra chuẩn hóa lại.

3.2.1 Bao đóng và tính tối thiểu

Với USER, $\{\text{UserID}\}^+ = R_{\text{User}}$ và $\{\text{Email}\}^+ = R_{\text{User}}$. Không có phụ thuộc từ tập rỗng, nên cả hai tập đơn đều tối thiểu và là khóa ứng viên.

Với CREDIT, $\{\text{CreditID}\}^+ = R_{\text{Credit}}$ và $\{\text{CustomerID}, \text{MerchantID}\}^+ = R_{\text{Credit}}$. CustomerID riêng không xác định ví vì một khách có ví ở nhiều Merchant; MerchantID riêng không xác định ví vì một cửa hàng phục vụ nhiều khách. Do đó cặp là khóa ứng viên tối thiểu. CreditID được chọn làm PK để giao dịch tham chiếu gọn hơn; UNIQUE trên cặp vẫn bắt buộc.

Với API_CREDENTIAL, KeyID và ApiKey là hai khóa ứng viên đơn, mỗi khóa có bao đóng bằng toàn quan hệ. Với các bảng còn lại, ID ở dòng tương ứng có bao đóng bằng toàn quan hệ và là khóa ứng viên đơn theo tập phụ thuộc hàm đã xét.

3.3 Chuẩn hóa Dữ liệu

Một quan hệ đạt BCNF nếu mọi phụ thuộc hàm không tầm thường $X \rightarrow Y$ có X là siêu khóa. Trong tập phụ thuộc trên, tất cả vế trái đều là khóa ứng viên của chính quan hệ đó. Theo tập phụ thuộc này, mười quan hệ đáp ứng BCNF.

Kết luận này giảm đi thường cập nhật do các phụ thuộc hàm đã xét, không thay thế ràng buộc miền giá trị, kiểm soát đồng thời hoặc toàn vẹn liên bảng. AvailableBalance là số dư lưu sẵn để tra cứu; phải được tăng dịch vụ duy trì nhất quán với sổ cái, không thể suy ra tính nhất quán đó chỉ từ BCNF.

3.3.1 Cách kiểm tra dạng chuẩn

1NF yêu cầu mỗi ô chứa một giá trị thuộc miền đã chọn, không chứa danh sách lặp. 2NF yêu cầu đạt 1NF và thuộc tính không khóa không phụ thuộc vào một phần thực sự của khóa ứng viên. 3NF yêu cầu với mỗi phụ thuộc hàm không tầm thường, về trái là siêu khóa hoặc thuộc tính về phải là thuộc tính khóa. BCNF yêu cầu về trái luôn là siêu khóa.

Ví dụ, nếu đưa TenantName vào MERCHANT thì xuất hiện MerchantID xác định TenantID và TenantID xác định TenantName. Một tổ chức có nhiều cửa hàng sẽ khiến tên tổ chức bị lưu lặp. Giữ TenantName ở TENANT và tham chiếu bằng TenantID giúp tránh cập nhật tên ở nhiều dòng MERCHANT.

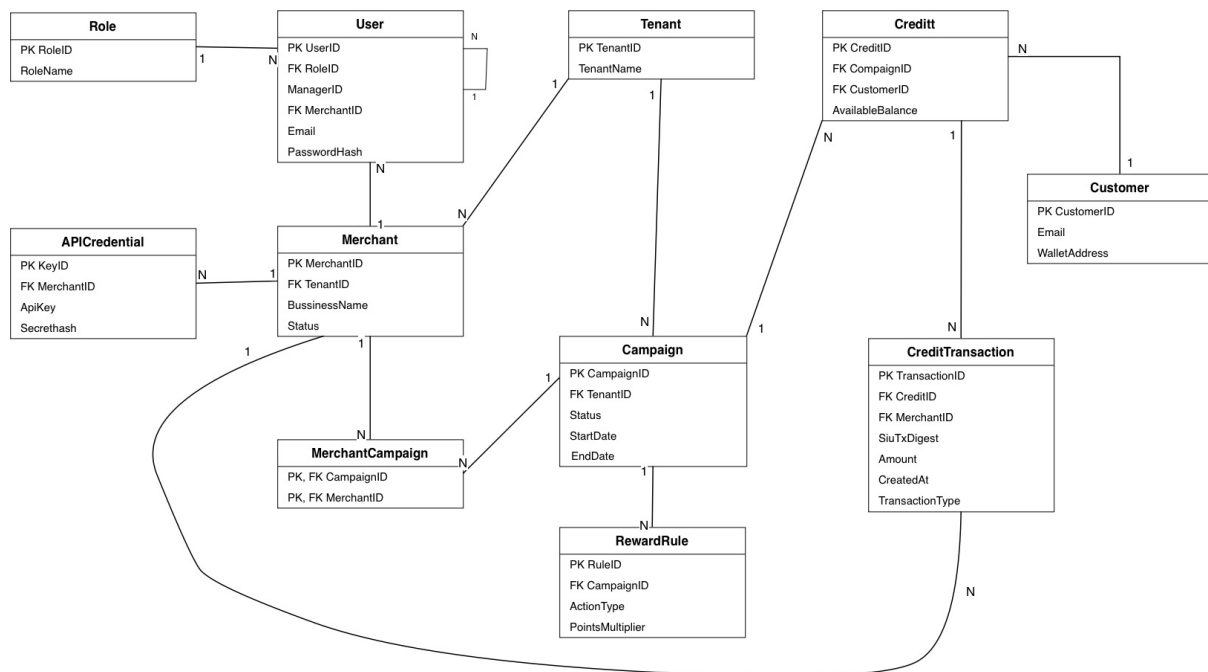
3.4 Sơ đồ Lược đồ Logic (Logical Schema Diagram)

Lược đồ dưới đây tương ứng với tập phụ thuộc hàm đã xét ở mục chuẩn hóa. Các ràng buộc miền giá trị và duy nhất bổ sung được trình bày ở từ điển dữ liệu.

Đặc biệt, hệ thống tồn tại mối kết hợp Nhiều - Nhiều (M:N) giữa CUSTOMER và MERCHANT. Mỗi kết hợp này đã được phân rã thành hai mối quan hệ 1-N thông qua thực thể trung gian (Associative Entity) là bảng CREDIT.

Dưới đây là Sơ đồ Logic thể hiện trực quan cấu trúc các bảng, tập hợp đầy đủ các thuộc tính, khóa chính (Primary Key - PK) và các mối quan hệ khóa ngoại (Foreign Key - FK) kèm bản số (1-N) liên kết giữa các bảng.

3.4.1 Sơ đồ thiết kế Logic



Hình 3.1: Sơ đồ thiết kế logic (Logical Diagram)

3.5 Từ điển Dữ liệu (Data Dictionary)

Từ điển dữ liệu (Data Dictionary) mô tả chi tiết cấu trúc vật lý của các bảng trong cơ sở dữ liệu. Nó bao gồm tên các thuộc tính, kiểu dữ liệu, các ràng buộc toàn vẹn (Khóa chính - PK, Khóa ngoại - FK, Not Null) và diễn giải ý nghĩa của từng trường dữ liệu để chuẩn bị cho bước triển khai vật lý ở Chương 4.

3.5.1 Bảng TENANT (Đơn vị thuê bao)

Lưu trữ thông tin về các đơn vị khách hàng lớn thuê hệ thống OctaPoint CaaS.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
TenantID	VARCHAR(36)	PK, NOT NULL	Mã định danh duy nhất (UUID).
TenantName	NVARCHAR(255)	NOT NULL	Tên định danh của đơn vị thuê bao.

3.5.2 Bảng MERCHANT (Doanh nghiệp đối tác)

Lưu trữ thông tin các doanh nghiệp/cửa hàng trực thuộc một Tenant.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MerchantID	VARCHAR(36)	PK, NOT NULL	Mã định danh duy nhất của Merchant.
TenantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng TENANT.
BusinessName	NVARCHAR(255)	NOT NULL	Tên thương hiệu/doanh nghiệp.
Status	VARCHAR(20)	NOT NULL	Trạng thái hoạt động (Active, Inactive).

3.5.3 Bảng ROLE (Vai trò)

Lưu trữ thông tin các nhóm quyền hạn để phân quyền người dùng trong hệ thống.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
RoleID	INT	PK, NOT NULL	Mã định danh vai trò.
RoleName	NVARCHAR(50)	NOT NULL	Tên hiển thị của vai trò (VD: Admin, Staff).

3.5.4 Bảng USER (Tài khoản nhân sự)

Quản lý tài khoản đăng nhập của nhân viên thuộc các Merchant.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
UserID	VARCHAR(36)	PK, NOT NULL	Mã định danh tài khoản.
RoleID	INT	FK, NOT NULL	Mã quyền hạn của người dùng.
ManagerID	VARCHAR(36)	FK, NULL	Mã người quản lý trực tiếp (tham chiếu đến UserID).
MerchantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng MERCHANT.
Email	VARCHAR(255)	NOT NULL	Email đăng nhập (duy nhất).
PasswordHash	VARCHAR(255)	NOT NULL	Chuỗi băm mật khẩu bảo mật.

3.5.5 Bảng API_CREDENTIAL (Khóa tích hợp)

Cung cấp khóa API để tích hợp phần mềm bán hàng của cửa hàng với nền tảng.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
KeyID	VARCHAR(36)	PK, NOT NULL	Mã định danh khóa.
MerchantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng MERCHANT.
ApiKey	VARCHAR(255)	NOT NULL	Mã định danh khóa công khai.
SecretHash	VARCHAR(255)	NOT NULL	Chuỗi băm của Secret Key.

3.5.6 Bảng CAMPAIGN (Chiến dịch)

Quản lý các chương trình khuyến mãi, tích/đổi điểm.

3.5.7 Bảng MERCHANT_CAMPAIGN (Phạm vi áp dụng)

Bảng trung gian quy định chiến dịch nào được phép chạy ở cửa hàng nào.

3.5.8 Bảng REWARD_RULE (Quy tắc tính điểm)

Chi tiết cấu hình quy tắc và hệ số nhân điểm của từng chiến dịch.

3.5.9 Bảng CUSTOMER (Khách hàng)

Quản lý thông tin định danh của khách hàng tích điểm.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CampaignID	VARCHAR(36)	PK, NOT NULL	Mã định danh chiến dịch.
TenantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng TENANT.
Status	VARCHAR(20)	NOT NULL	Trạng thái (Draft, Active, Ended).
StartDate	DATETIME	NOT NULL	Thời gian bắt đầu.
EndDate	DATETIME	NULL	Thời điểm kết thúc.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CampaignID	VARCHAR(36)	PK, FK, NOT NULL	Tham chiếu chiến dịch.
MerchantID	VARCHAR(36)	PK, FK, NOT NULL	Tham chiếu cửa hàng.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
RuleID	VARCHAR(36)	PK, NOT NULL	Mã định danh quy tắc.
CampaignID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến chiến dịch.
ActionType	VARCHAR(50)	NOT NULL	Loại hành động (VD: Mua hàng).
PointsMultiplier	DECIMAL(5,2)	NOT NULL	Hệ số nhân điểm thưởng.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CustomerID	VARCHAR(36)	PK, NOT NULL	Mã định danh khách hàng nội bộ.
Email	VARCHAR(255)	NOT NULL	Địa chỉ email (Định danh Web2).
WalletAddress	VARCHAR(100)	NULL	Địa chỉ ví blockchain (Sui).

3.5.10 Bảng CREDIT (Số dư ví điểm)

Lưu trữ số dư khả dụng của khách hàng theo từng chiến dịch.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CreditID	VARCHAR(36)	PK, NOT NULL	Mã định danh bản ghi số dư.
CustomerID	VARCHAR(36)	FK, NOT NULL	Tham chiếu khách hàng.
CampaignID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến chiến dịch phát hành điểm.
AvailableBalance	DECIMAL(18,2)	NOT NULL	Số dư khả dụng hiện tại.

3.5.11 Bảng CREDIT_TRANSACTION (Số cái giao dịch điểm)

Lưu vết toàn bộ các giao dịch cộng/trừ điểm thưởng, có lưu băm trên blockchain.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
TransactionID	VARCHAR(36)	PK, NOT NULL	Mã giao dịch định danh.
CreditID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến ví điểm.
MerchantID	VARCHAR(36)	FK, NOT NULL	Cửa hàng phát sinh giao dịch.
Amount	DECIMAL(18,2)	NOT NULL	Số lượng điểm biến động.
CreatedAt	DATETIME	NOT NULL	Thời gian phát sinh giao dịch.
TransactionType	VARCHAR(50)	NOT NULL	Loại giao dịch (Earn, Burn).
SuiTxDigest	VARCHAR(100)	NULL	Mã băm trên mạng lưới Sui.

Chương 4

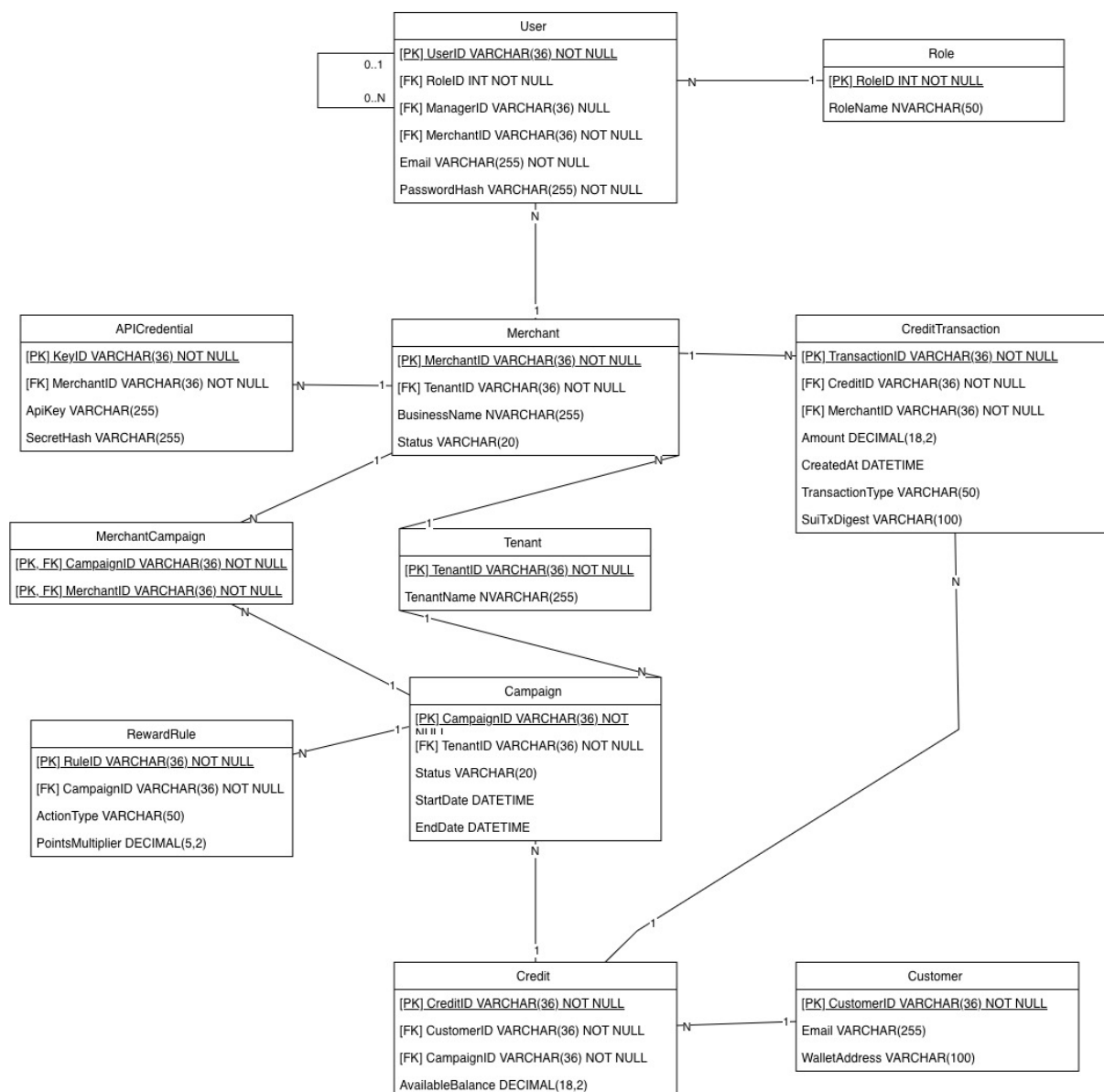
Thiết kế Vật lý và Triển khai

Theo quy trình thiết kế cơ sở dữ liệu, thiết kế vật lý không làm thay đổi ý nghĩa dữ liệu mà quyết định cách Hệ quản trị CSDL (DBMS) lưu trữ, đọc, ghi và tìm kiếm dữ liệu tối ưu nhất dựa trên tải công việc (workload).

Chương này sử dụng lược đồ logic đã chuẩn hóa và các đặc tả workload làm đầu vào, từ đó tiến hành lựa chọn tổ chức lưu trữ, đánh chỉ mục và xuất ra cấu trúc mã lệnh DDL vật lý.

4.0.1 Chọn hệ quản trị cơ sở dữ liệu

Hệ thống sử dụng hệ quản trị cơ sở dữ liệu SQL Server. Đây là một hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, hỗ trợ tốt cho việc xử lý các giao dịch phức tạp, tính toàn vẹn dữ liệu cao và dễ dàng tích hợp với các hệ thống doanh nghiệp.



Hình 4.1: Sơ đồ thiết kế vật lý (Physical Diagram)

4.1 Ánh xạ Kiểu dữ liệu và Ràng buộc

Theo quy trình xử lý, đầu vào của thiết kế vật lý bao gồm Lược đồ logic đã được chuẩn hóa (đạt chuẩn BCNF ở Chương 3), cùng với tập hợp các khóa và ràng buộc. Hệ thống OctaPoint tiến hành ánh xạ các yếu tố này sang hệ quản trị SQL Server như sau:

- **VARCHAR(36)**: Sử dụng để lưu trữ các khóa chính/khóa ngoại dạng UUID/GUID.
- **NVARCHAR(n)**: Sử dụng cho các trường văn bản có chứa ký tự tiếng Việt hoặc Unicode (ví dụ: *RoleName*, *BusinessName*, *TenantName*).
- **VARCHAR(n)**: Sử dụng cho các trường văn bản không chứa dấu (ví dụ: *Email*, *PasswordHash*, *ApiKey*).
- **INT**: Sử dụng cho các số nguyên tính (ví dụ: *RoleID*).

- **DECIMAL(p, s)**: Sử dụng cho dữ liệu số có phần thập phân cần độ chính xác cao như số dư (*AvailableBalance*), số tiền (*Amount*) hoặc hệ số (*PointsMultiplier*).
- **DATETIME**: Sử dụng để lưu trữ thông tin ngày giờ (ví dụ: *StartDate*, *EndDate*, *CreatedAt*).

Các ràng buộc tính toàn vẹn (Integrity Constraints) bao gồm: **PRIMARY KEY** để định danh thực thể, **FOREIGN KEY** để duy trì tính toàn vẹn tham chiếu, và **NOT NULL** cho các trường thông tin bắt buộc.

4.2 Chiến lược Đánh chỉ mục

Chỉ mục giảm chi phí đọc ở một số truy vấn nhưng tăng dung lượng và chi phí ghi. Việc lựa chọn phải dựa trên bộ lọc, phép nối, thứ tự kết quả và số liệu thực thi.

4.2.1 Chỉ mục triển khai

- Mọi PK được khai báo **CLUSTERED** trong `database/schema.sql`. **UNIQUE** tạo chỉ mục hỗ trợ các khóa nghiệp vụ; không tạo thêm chỉ mục Email trùng chức năng.
- SQL Server không tự tạo chỉ mục cho FK. DDL bổ sung chỉ mục cho TenantID của MERCHANT; MerchantID, RoleID, ManagerID của USER; MerchantID của API_CREDENTIAL, CAMPAIGN, CREDIT; CampaignID của REWARD_RULE và CREDIT_TRANSACTION.
- CREDIT dùng **UNIQUE (CustomerID, MerchantID)**, đồng thời hỗ trợ tra cứu theo CustomerID ở cột dẫn đầu.
- Chỉ mục (**CreditID, CreatedAt DESC**) hỗ trợ lọc lịch sử một ví theo thời gian. Không tạo riêng chỉ mục CreditID trùng tiền tố.
- Hai chỉ mục duy nhất có điều kiện bảo vệ WalletAddress và SuiTxDigest khi khác NULL.

4.2.2 Đánh giá và giới hạn

Tỷ lệ $NDV(A)/|R|$ biểu diễn mức phân biệt giá trị, không phải tỷ lệ dòng thỏa một điều kiện cụ thể. Hiệu quả chỉ mục còn phụ thuộc phân bố dữ liệu, số dòng trả về và chi phí truy cập.

Chưa tạo chỉ mục đơn trên Status (Merchant: Active/Suspended; Campaign: Draft/Active/Expired) hoặc TransactionType (Earn/Redeem/Refund). Miền giá trị nhỏ không có nghĩa chỉ mục luôn vô ích: chỉ mục ghép hoặc có điều kiện vẫn có thể hữu ích với workload phù hợp. Cần kiểm tra execution plan, statistics và chi phí ghi trước khi bổ sung.

4.2.3 Mã lệnh khởi tạo cơ sở dữ liệu (T-SQL)

Listing 4.1: Script khởi tạo các bảng trên SQL Server

— 1. Tạo bảng Role (Tạo trước vì User tham chiếu đến)

```
CREATE TABLE Role (
```

```

        RoleID INT NOT NULL PRIMARY KEY,
        RoleName NVARCHAR(50)
    );

-- 2. Tao bang Tenant
CREATE TABLE Tenant (
    TenantID VARCHAR(36) NOT NULL PRIMARY KEY,
    TenantName NVARCHAR(255)
);

-- 3. Tao bang Customer
CREATE TABLE Customer (
    CustomerID VARCHAR(36) NOT NULL PRIMARY KEY,
    Email VARCHAR(255),
    WalletAddress VARCHAR(100)
);

-- 4. Tao bang Merchant
CREATE TABLE Merchant (
    MerchantID VARCHAR(36) NOT NULL PRIMARY KEY,
    TenantID VARCHAR(36) NOT NULL,
    BusinessName NVARCHAR(255),
    Status VARCHAR(20),
    CONSTRAINT FK_Merchant_Tenant FOREIGN KEY (TenantID) REFERENCES Tenant(
);

-- 5. Tao bang User (Luu y tu khoa [User] trong SQL Server)
CREATE TABLE [User] (
    UserID VARCHAR(36) NOT NULL PRIMARY KEY,
    RoleID INT NOT NULL,
    ManagerID VARCHAR(36) NULL, -- 0..1 relationship
    MerchantID VARCHAR(36) NOT NULL,
    Email VARCHAR(255) NOT NULL,
    PasswordHash VARCHAR(255) NOT NULL,
    CONSTRAINT FK_User_Role FOREIGN KEY (RoleID) REFERENCES Role(RoleID),
    CONSTRAINT FK_User_Manager FOREIGN KEY (ManagerID) REFERENCES [User](U
    CONSTRAINT FK_User_Merchant FOREIGN KEY (MerchantID) REFERENCES Merchant
);

-- 6. Tao bang APICredential
CREATE TABLE APICredential (
    KeyID VARCHAR(36) NOT NULL PRIMARY KEY,
    MerchantID VARCHAR(36) NOT NULL,
    ApiKey VARCHAR(255),
    SecretHash VARCHAR(255),
    CONSTRAINT FK_APICredential_Merchant FOREIGN KEY (MerchantID) REFERENC
);

```

-- 7. Tao bang Campaign

```
CREATE TABLE Campaign (  
    CampaignID VARCHAR(36) NOT NULL PRIMARY KEY,  
    TenantID VARCHAR(36) NOT NULL,  
    Status VARCHAR(20),  
    StartDate DATETIME,  
    EndDate DATETIME,  
    CONSTRAINT FK_Campaign_Tenant FOREIGN KEY (TenantID) REFERENCES Tenant(  
);
```

-- 8. Tao bang MerchantCampaign (Bang trung gian n-n)

```
CREATE TABLE MerchantCampaign (  
    CampaignID VARCHAR(36) NOT NULL,  
    MerchantID VARCHAR(36) NOT NULL,  
    PRIMARY KEY (CampaignID, MerchantID),  
    CONSTRAINT FK_MC_Campaign FOREIGN KEY (CampaignID) REFERENCES Campaign(  
    CONSTRAINT FK_MC_Merchant FOREIGN KEY (MerchantID) REFERENCES Merchant(  
);
```

-- 9. Tao bang RewardRule

```
CREATE TABLE RewardRule (  
    RuleID VARCHAR(36) NOT NULL PRIMARY KEY,  
    CampaignID VARCHAR(36) NOT NULL,  
    ActionType VARCHAR(50),  
    PointsMultiplier DECIMAL(5,2),  
    CONSTRAINT FK_RewardRule_Campaign FOREIGN KEY (CampaignID) REFERENCES C  
);
```

-- 10. Tao bang Credit

```
CREATE TABLE Credit (  
    CreditID VARCHAR(36) NOT NULL PRIMARY KEY,  
    CustomerID VARCHAR(36) NOT NULL,  
    CampaignID VARCHAR(36) NOT NULL,  
    AvailableBalance DECIMAL(18,2),  
    CONSTRAINT FK_Credit_Customer FOREIGN KEY (CustomerID) REFERENCES Custo  
    CONSTRAINT FK_Credit_Campaign FOREIGN KEY (CampaignID) REFERENCES Camp  
);
```

-- 11. Tao bang CreditTransaction

```
CREATE TABLE CreditTransaction (  
    TransactionID VARCHAR(36) NOT NULL PRIMARY KEY,  
    CreditID VARCHAR(36) NOT NULL,  
    MerchantID VARCHAR(36) NOT NULL,  
    Amount DECIMAL(18,2),  
    CreatedAt DATETIME,  
    TransactionType VARCHAR(50),  
    SuiTxDigest VARCHAR(100),  
    CONSTRAINT FK_Transaction_Credit FOREIGN KEY (CreditID) REFERENCES Cre
```

CONSTRAINT FK_Transaction_Merchant FOREIGN KEY (MerchantID) REFERENCES
);

4.3 Tối ưu hóa truy xuất dữ liệu

Dựa trên pipeline xử lý truy vấn của hệ quản trị CSDL, việc viết câu lệnh SQL là chưa đủ. Hệ thống cần tối ưu hóa cây truy vấn (Query Tree) thông qua các luật biến đổi tương đương để giảm thiểu chi phí bộ nhớ và I/O trước khi lập kế hoạch thực thi (Execution Plan).

4.3.1 Bài toán truy vấn

Yêu cầu: Lấy ra Mã giao dịch (TransactionID) và Số điểm (Amount) của các giao dịch cộng điểm lớn hơn 500, thuộc về các Chiến dịch (CAMPAIGN) đang ở trạng thái hoạt động ('Active').

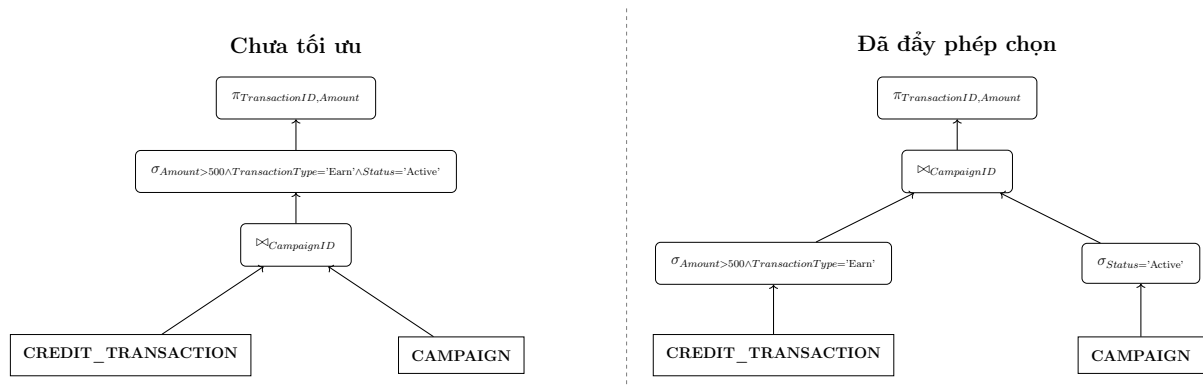
Biểu thức đại số quan hệ:

$$\begin{aligned} T' &= \sigma_{Amount > 500 \wedge TransactionType = 'Earn'}(T), \\ C' &= \sigma_{Status = 'Active'}(C), \\ Q &= \pi_{TransactionID, Amount}(T' \bowtie_{T'.CampaignID = C'.CampaignID} C'). \end{aligned}$$

Trong đó T là CREDIT_TRANSACTION, C là CAMPAIGN. Phép nối dùng điều kiện bằng CampaignID, không phải nối tự nhiên trên tất cả thuộc tính cùng tên.

4.3.2 Cây truy vấn trước và sau khi tối ưu

Theo luật biến đổi tương đương, phép chọn (Selection - σ) nếu được đẩy xuống trước phép kết nối (Join - $\bowtie$) sẽ giúp giảm thiểu đáng kể kích thước dữ liệu trung gian.



Hình 4.2: Cây truy vấn trước và sau khi tối ưu hóa bằng luật phân rã

4.3.3 Câu lệnh SQL tương ứng

```
1 SELECT ct.TransactionID, ct.Amount
2 FROM CREDIT_TRANSACTION AS ct
3 JOIN CAMPAIGN AS c ON c.CampaignID = ct.CampaignID
```

```

4 WHERE ct.Amount > 500
5     AND ct.TransactionType = 'Earn'
6     AND c.Status = 'Active';

```

4.3.4 Đánh giá thuật toán kết nối

Đẩy phép chọn xuống có thể giảm số dòng trung gian; mức giảm thực tế phụ thuộc phân bố dữ liệu. SQL Server chọn Nested Loops, Hash hoặc Merge Join theo statistics, chỉ mục và chi phí ước lượng. Nested Loops thường phù hợp khi đầu vào ngoài nhỏ và phía trong có đường tra cứu hiệu quả; không thể kết luận chỉ từ việc CAMPAIGN nhỏ.

Đồ án chưa có execution plan hoặc số đo I/O, thời gian trước/sau. Vì vậy đây là phân tích biến đổi logic, không phải bằng chứng một thuật toán luôn được chọn hay truy vấn đã đạt mục tiêu hiệu năng.

4.4 Một số câu truy vấn mẫu

Dưới đây là các câu truy vấn T-SQL tiêu biểu để kiểm chứng thiết kế cơ sở dữ liệu trên hệ quản trị SQL Server.

4.4.1 Truy vấn 1: Xem lịch sử giao dịch điểm của một khách hàng tại một cửa hàng

```

1 SELECT
2     ct.TransactionID,
3     ct.TransactionType,
4     ct.Amount,
5     ct.CreatedAt
6 FROM CREDIT_TRANSACTION ct
7 JOIN CREDIT c ON ct.CreditID = c.CreditID
8 WHERE c.CustomerID = 'CUST_UUID_001'
9     AND c.MerchantID = 'MERCH_UUID_002'
10 ORDER BY ct.CreatedAt DESC, ct.TransactionID DESC;

```

4.4.2 Truy vấn 2: Tra cứu số dư điểm hiện tại của khách hàng tại các cửa hàng đang hoạt động

```

1 SELECT
2     m.BusinessName,
3     c.AvailableBalance
4 FROM CREDIT c
5 JOIN MERCHANT m ON c.MerchantID = m.MerchantID
6 WHERE c.CustomerID = 'CUST_UUID_001'
7     AND m.Status = 'Active'
8 ORDER BY c.AvailableBalance DESC, c.MerchantID;

```

4.4.3 Truy vấn 3: Thống kê tổng số điểm đã phát hành theo từng chiến dịch có phát sinh tích điểm trong tháng hiện tại

```
1 DECLARE @MonthStart DATETIME = DATEFROMPARTS(YEAR(GETDATE()), MONTH(  
2     GETDATE()), 1);  
3  
4 SELECT  
5     camp.CampaignName,  
6     COUNT(ct.TransactionID) AS TotalTransactions,  
7     SUM(ct.Amount) AS TotalPointsIssued  
8 FROM CREDIT_TRANSACTION ct  
9 JOIN CAMPAIGN camp ON ct.CampaignID = camp.CampaignID  
10 WHERE camp.MerchantID = 'MERCH_UUID_002'  
11     AND ct.TransactionType = 'Earn'  
12     AND ct.CreatedAt >= @MonthStart  
13     AND ct.CreatedAt < DATEADD(MONTH, 1, @MonthStart)  
14 GROUP BY camp.CampaignID, camp.CampaignName;
```

Khoảng thời gian nửa kín ở truy vấn 3 giữ cột CreatedAt nguyên dạng để bộ tối ưu có thể tận dụng chỉ mục phù hợp. Truy vấn hiện có chưa bảo đảm kế hoạch seek; cần kiểm tra execution plan.

Tổng điểm phát hành ở truy vấn 3 chỉ cộng giao dịch Earn, chưa trừ các giao dịch hoàn trả. Chiến dịch không có Earn trong tháng không xuất hiện do sử dụng INNER JOIN. Các mã khách hàng và cửa hàng trong ví dụ cần được thay bằng mã tồn tại trong dữ liệu thử.